Tutorial 1: Markov Decision Processes

Reinforcement Learning Course

Learning Objectives

By the end of this tutorial, you should be able to:

- Formulate sequential decision problems as MDPs
- Decide whether a state representation satisfies the Markov property
- Compute discounted returns
- Derive the Bellman equation and use it to compute V^π and Q^π exactly for small MDPs
- Relate V^*, Q^* and optimal policies, and recognise the effect of reward design on optimal behaviour

Notation Reference

- $\mathcal{S}$: State space, $s \in \mathcal{S}$: a state
- $\mathcal{A}$: Action space, $a \in \mathcal{A}$: an action
- $P(s'|s,a)$: Transition probability
- $P(s',r|s,a)$: Transition and reward probability
- $R(s,a,s')$: Reward function
- $\pi(a|s)$: Policy (probability of action a in state s)
- $V^\pi(s)$: State value function under policy π
- $Q^\pi(s,a)$: Action value function under policy π
- $V^*(s)$, $Q^*(s,a)$: Optimal state and action value functions
- γ: Discount factor, $0 \leq \gamma < 1$

1 Understanding MDPs

1.1 MDP Formulation

Consider a simple robot navigation problem: A robot is on a 1D track with 5 positions labeled 0, 1, 2, 3, 4. Position 0 is the starting position, and position 4 is the goal (but not terminal). The robot can take actions: `LEFT`, `RIGHT`, or `STAY`.

Let's build the complete MDP formulation $(\mathcal{S}, \mathcal{A}, P, R)$ step by step:

(a) **State and Action Spaces:** Define the state space $\mathcal{S}$ and action space $\mathcal{A}$ for this problem. Be explicit about what each represents.

(b) **Transition Function:** Assume deterministic transitions (the robot always moves as intended). Define the transition function $P(s'|s, a)$ and write it out explicitly for at least 3 state-action pairs. What happens at the boundaries?

(c) **Reward Function:** Design a reward function $R(s, a, s')$ where: reaching the goal gives $+10$, each step costs -1, and staying in place when not at the goal costs -2. Write out $R(s, a, s')$ for at least 3 cases.

1.2 The Markov Property

For each of the following, state whether the given state representation satisfies the Markov property. If it does not, suggest how the state could be augmented so that it (approximately) does.

(a) Pong, where the state is a single frame of the screen.

(b) Pong, where the state is the last 4 frames of the screen stacked together.

(c) A robot arm, where the state is its joint angles only.

(d) A robot arm, where the state is its joint angles and joint velocities.

(e) Texas Hold'em poker, where the state is your own two cards and the cards on the table.

(f) Chess, where the state is the position of the pieces on the board.

1.3 Computing Returns

Consider a sequence of rewards: $R_1 = 1$, $R_2 = 2$, $R_3 = 5$, $R_4 = 0$.

(a) Calculate the discounted return G_0 with $\gamma = 0.9$.

(b) Calculate the discounted return G_0 with $\gamma = 0.5$.

(c) What is the return with $\gamma = 0$ (myopic agent)?

(d) Explain intuitively why we use discounting in RL.

(e) Now consider a *continuing* task in which the agent receives a reward of $+1$ at every time step, forever. Calculate G_0 with $\gamma = 0.9$. Give a general expression for a constant reward r and discount γ.

(f) What happens to G_0 in part (e) if $\gamma = 1$? Why is $\gamma = 1$ acceptable for episodic tasks but not for continuing ones?

2 Bellman Equation Derivation

Starting from the definition of the state value function:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

where $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$

(a) Show that $G_t = R_{t+1} + \gamma G_{t+1}$.

(b) Derive the Bellman equation:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} P(s', r|s, a)[r + \gamma V^\pi(s')]$$

(c) Explain in words what this equation means. Why is it recursive?

3 Solving the Bellman Equation for a Fixed Policy

3.1 Exact Policy Evaluation

Consider a 2-state MDP with states $\{s_1, s_2\}$, actions $\{a_1, a_2\}$ and discount factor $\gamma = 0.9$. The dynamics are:

s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
s_1	a_1	s_1	0.6	4
s_1	a_1	s_2	0.4	1
s_1	a_2	s_2	1.0	3
s_2	a_1	s_1	1.0	1
s_2	a_2	s_1	0.2	-2
s_2	a_2	s_2	0.8	0

The agent follows the deterministic policy $\pi(s_1) = a_1$, $\pi(s_2) = a_2$.

(a) Compute the expected immediate reward $r_\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) R(s,a,s')$ for each state, and the state transition matrix P_π under π.

(b) Write down the Bellman equation for $V^\pi(s_1)$ and $V^\pi(s_2)$. This gives two linear equations in two unknowns; solve them exactly.

(c) Write the Bellman equation in matrix form and give the closed-form solution for V^π. Why is this approach impractical for MDPs with many states?

(d) Compute $Q^\pi(s,a)$ for all four state-action pairs, using

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s')\right].$$

Check that $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$ holds in each state.

(e) Compare $Q^\pi(s_2, a_1)$ with $V^\pi(s_2)$. What does this suggest about the policy π?

4 Optimal Value Functions and Policies

4.1 From Q^* to V^* and π^*

You are given the optimal action-value function of an MDP with three states and three actions:

$Q^*(s,a)$	a_1	a_2	a_3
s_1	4.0	6.5	6.5
s_2	3.2	1.0	2.8
s_3	-1.0	-0.5	-2.0

(a) Compute $V^*(s)$ for each state.

(b) Write down an optimal deterministic policy π^*. Is it unique?

(c) Is the stochastic policy that picks a_2 or a_3 with probability 0.5 each in s_1, and acts as in (b) elsewhere, optimal?

(d) Is a policy that always takes a_1 in s_1 (and acts as in (b) elsewhere) optimal? Justify your answer using the table.

4.2 Optimal Values and Reward Design

Return to the 1D robot track from Section 1.1, with $\gamma = 0.9$. To remove any ambiguity, the reward function is:

- Moving `RIGHT` from position 3 into position 4: $+9$ (the $+10$ goal reward minus the step cost)
- Any other `LEFT` or `RIGHT` action (including bumping into a boundary): -1
- `STAY` at positions 0–3: -2; `STAY` at position 4: 0

(a) First suppose position 4 is a **terminal** state, so $V^*(4) = 0$. Working backwards from the goal, compute $V^*(s)$ for $s = 3, 2, 1, 0$.

(b) Still with position 4 terminal, use $Q^*(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma V^*(s')]$ to compute $Q^*(2,a)$ for all three actions, and state $\pi^*(2)$.

(c) Now suppose, as in the original problem, that position 4 is **not terminal**. Starting from position 3, compute the discounted return of each of the following policies:

(i) Move `RIGHT` into position 4, then `STAY` there forever.

(ii) Move `RIGHT` into position 4, then alternate `LEFT`, `RIGHT`, `LEFT`, `RIGHT`, ... forever.

(d) Which policy in (c) does an optimal agent prefer? What does this tell you about the reward function, and how could you fix it?